Analysis of Algorithm (CSN13400)
ASSIGNMENT-3

Name : Aayansh Niranjan Date :12/08/2026
Reg_No.: 20253004

Q1. Implementation and Analysis of Bubble Sort in C: Best, Worst, and Average Case Performance Using GNUplot. Implement Bubble Sort in C. Measure its execution time across best, worst, and average case input arrangements for varying input sizes n. Plot the resulting growth curves using GNUplot to analyze its empirical time complexity.

A1. CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void bubbleSort(int a[], int n){
    int i, j, temp;
    for(i=0;i<n-1;i++)
        for(j=0;j<n-i-1;j++)
            if(a[j]>a[j+1]){
                temp=a[j];
                a[j]=a[j+1];
                a[j+1]=temp;
            }
}

double getTime(int a[], int n){
    clock_t start,end;
    start=clock();
    bubbleSort(a,n);
    end=clock();
    return (double)(end-start)/CLOCKS_PER_SEC;
}

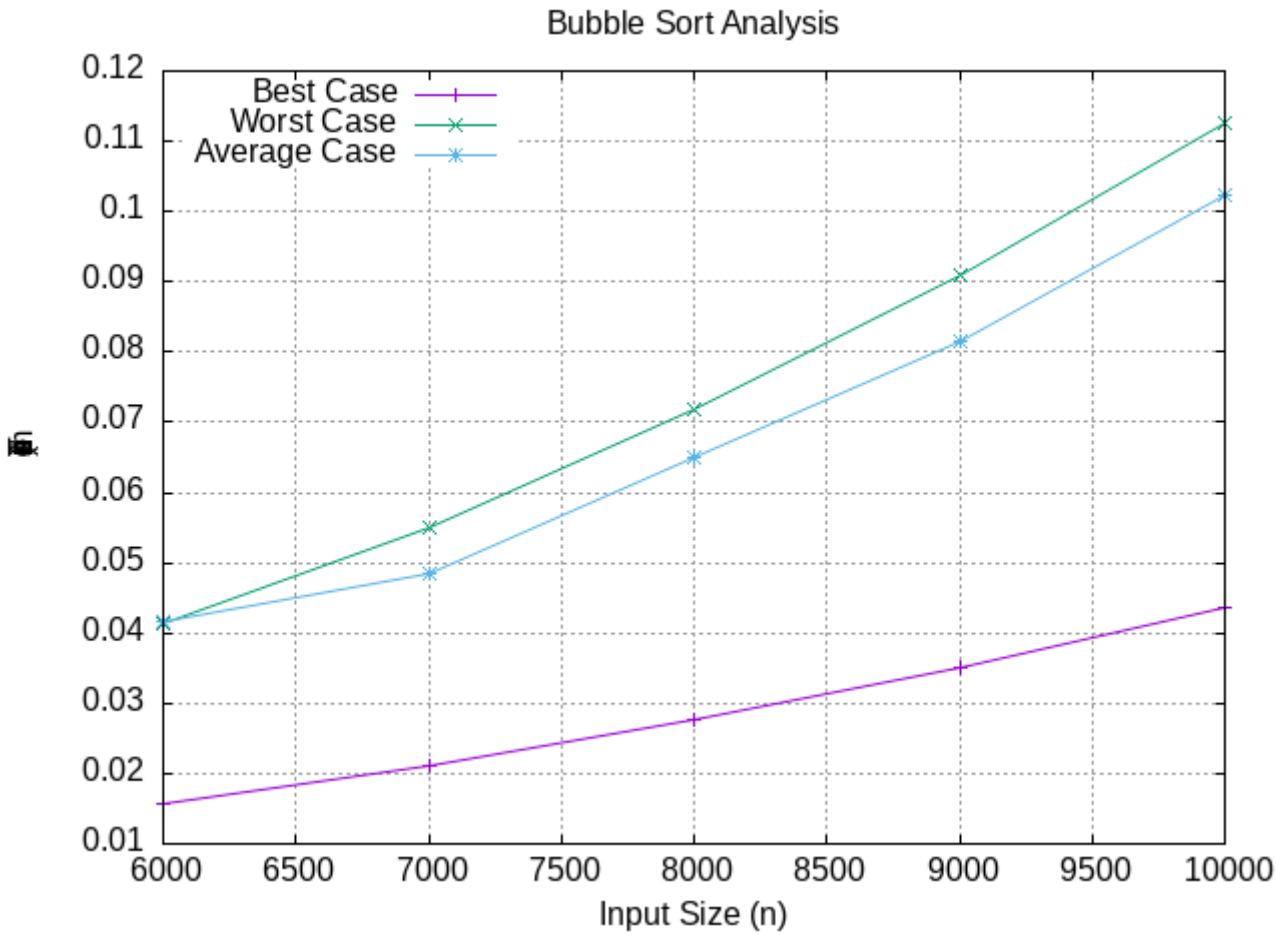
int main(){
  FILE *f=fopen("bubble.dat","w");
  srand(time(NULL));
  fprintf(f,"# n best worst average\n");
  for(int n=6000;n<=10000;n+=1000){
    int best[n],worst[n],avg[n];
    for(int i=0;i<n;i++){
        best[i]=i;
        worst[i]=n-i;
        avg[i]=rand()%1000;
    }
    fprintf(f,"%d %lf %lf %lf\n",n,getTime(best,n),getTime(worst,n),getTime(avg,n));
  }
  fclose(f);
  return 0;
}
```

OUTPUT:

DATA.TXT

```
# n best worst average
6000 0.015703 0.041250 0.041503
7000 0.021193 0.054930 0.048394
8000 0.027648 0.071800 0.064947
9000 0.034943 0.091033 0.081394
10000 0.043634 0.112575 0.102316
```

GRAPH



Q2. Implementation and Comparative Analysis of Standard Bubble Sort and Optimized Bubble Sort in C (Best Case) Using GNUplot. Implement both standard Bubble Sort and an optimized version (incorporating an early-exit flag for sorted arrays) in C. Measure and compare their execution times on best-case (pre sorted) input data across varying input sizes n. Use GNUplot to graph the performance comparison and demonstrate O(n) vs. O(n2) best-case behavior.

A2. CODE:

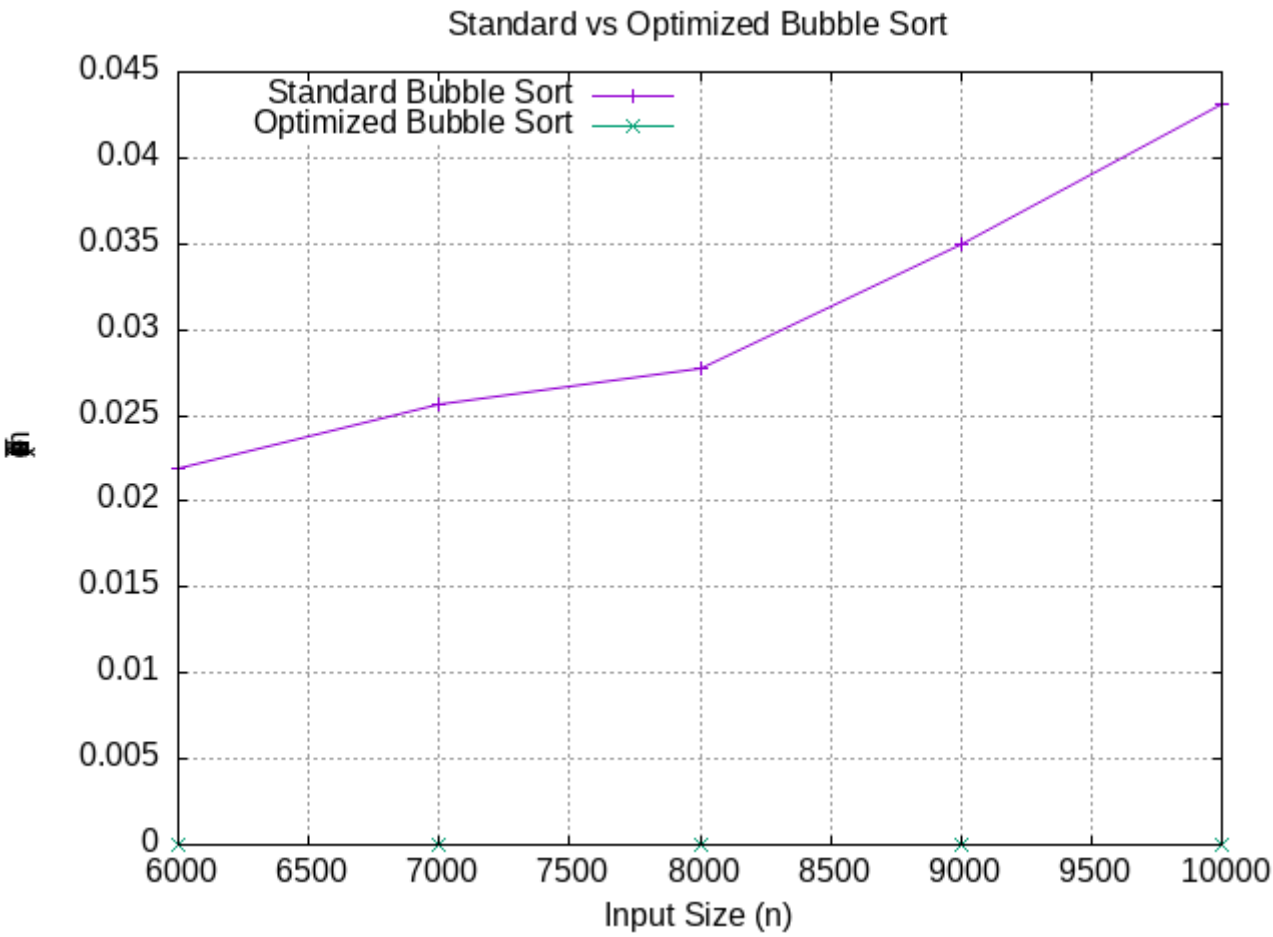
```
#include <stdio.h>
#include <time.h>
void bubbleSort(int a[], int n){
    int i,j,temp;
    for(i=0;i<n-1;i++)
        for(j=0;j<n-i-1;j++)
            if(a[j]>a[j+1]){
                temp=a[j];
                a[j]=a[j+1];
                a[j+1]=temp;
            }
}
void optimizedBubbleSort(int a[], int n){
    int i,j,temp,flag;
    for(i=0;i<n-1;i++){
        flag=0;
        for(j=0;j<n-i-1;j++){
            if(a[j]>a[j+1]){
                temp=a[j];
                a[j]=a[j+1];
                a[j+1]=temp;
                flag=1;
            }
        }
        if(flag==0){break;}
    }
}
double standardTime(int a[],int n){
    clock_t s,e;
    s=clock();
    bubbleSort(a,n);
    e=clock();
    return (double)(e-s)/CLOCKS_PER_SEC;
}
double optimizedTime(int a[],int n){
    clock_t s,e;
    s=clock();
    optimizedBubbleSort(a,n);
    e=clock();
    return (double)(e-s)/CLOCKS_PER_SEC;
}
int main(){
    FILE *f=fopen("compare.dat","w");
    fprintf(f,"# n standard optimized\n");
    for(int n=6000;n<=10000;n+=500){
        int a[n],b[n];
        for(int j=0;j<n;j++){a[j]=b[j]=j;}
        fprintf(f,"%d %lf %lf\n",n,standardTime(a,n),optimizedTime(b,n));
    }
    fclose(f);
    return 0;
}
```

OUTPUT:

DATA.TXT

```
# n standard optimized
6000 0.021884 0.000033
7000 0.025640 0.000008
8000 0.027719 0.000009
9000 0.034989 0.000010
10000 0.043100 0.000011
```

GRAPH



Q3. Implementation and Analysis of Insertion Sort in C: Best, Worst, and Average Case Performance Using GNUplot. Implement Insertion Sort in C. Measure and record execution times for best, worst, and average-case input configurations across varying input sizes n. Plot the execution time curves using GNUplot to analyze its theoretical vs. empirical performance
A3. CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void insertionSort(int a[], int n){
   int i,j,key;
   for(i=1;i<n;i++){
      key=a[i];
      j=i-1;
      while(j>=0 && a[j]>key){
         a[j+1]=a[j];
         j--;
      }
      a[j+1]=key;
   }
}

double getTime(int a[], int n){
   clock_t start,end;
   start=clock();
   insertionSort(a,n);
   end=clock();
   return (double)(end-start)/CLOCKS_PER_SEC;
}

int main(){
  FILE *f=fopen("insertion.dat","w");
  srand(time(NULL));
  fprintf(f,"# n best worst average\n");
  for(int n=6000;n<=10000;n+=1000){
    int best[n],worst[n],avg[n];
    for(int i=0;i<n;i++){
       best[i]=i;
       worst[i]=n-i;
       avg[i]=rand()%1000;
    }
    fprintf(f,"%d %lf %lf %lf\n",n,getTime(best,n),getTime(worst,n),getTime(avg,n));
  }
  fclose(f);
  return 0;
}
```

OUTPUT:

DATA.TXT

```
# n best worst average
6000 0.000012 0.022989 0.015635
7000 0.000014 0.029906 0.014909
8000 0.000015 0.039342 0.020111
9000 0.000017 0.049045 0.024701
10000 0.000017 0.061418 0.030501
```

GRAPH

